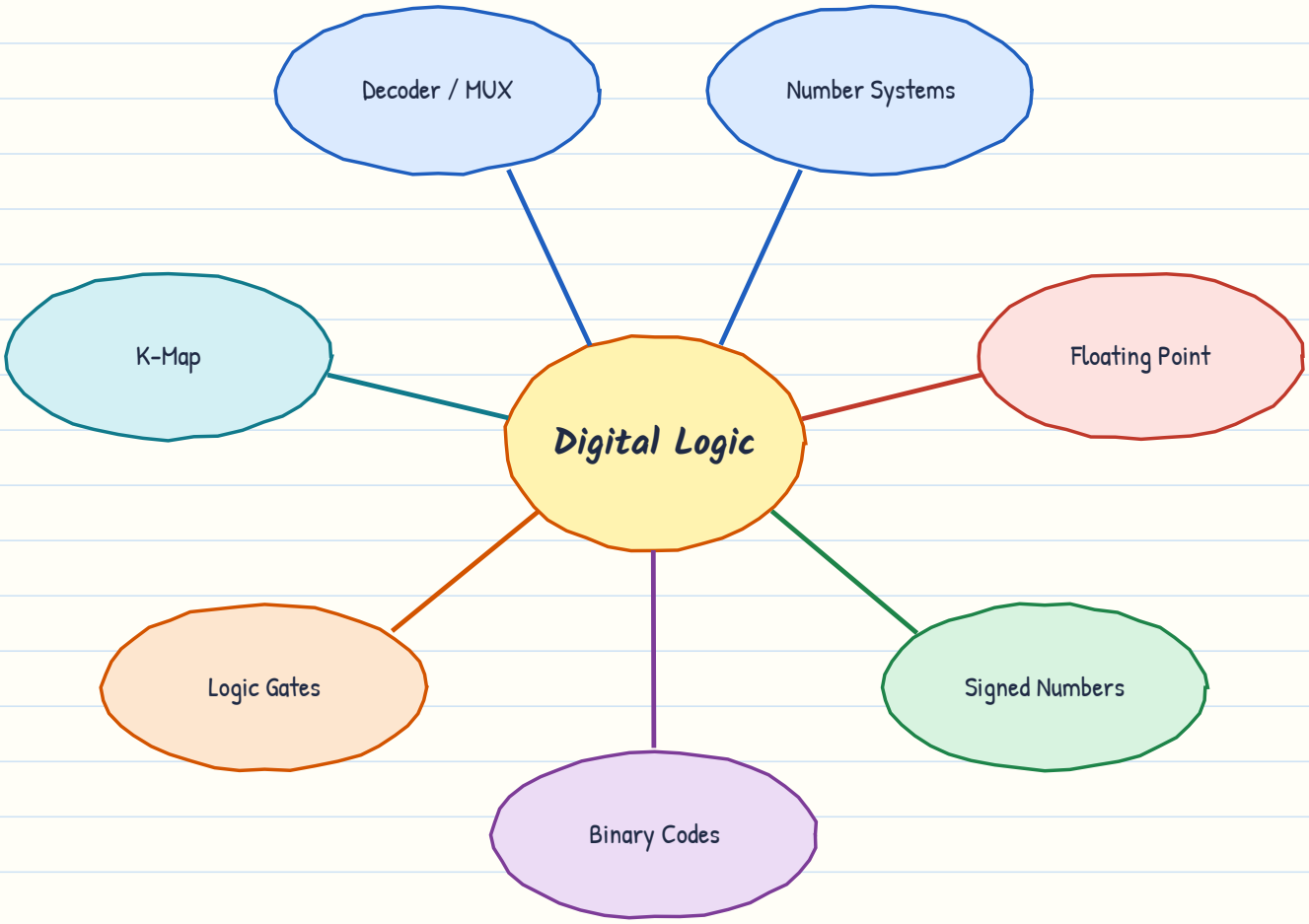
UNIT 1

Data Representation & Digital Logic Circuits-1

Number systems | Complements | Codes | Gates | K-map | Adders | Decoders | MUX



Mind map: everything you will learn in this unit

How to use these notes

Tip: first understand each diagram, then re-draw gates and K-maps on paper yourself - this is how digital logic is tested in exams.

1. Number Systems & Conversions

Computers store everything as **bits** (0 or 1). A number system is defined by its **base (radix)** - the count of digits it uses.

System	Base	Digits	Example
Binary	2	0, 1	1011
Octal	8	0 - 7	17
Decimal	10	0 - 9	45
Hexadecimal	16	0-9, A-F	2F

Dec	Bin	Oct	Hex	Dec	Bin	Oct	Hex
0	0000	0	0	8	1000	10	8
1	0001	1	1	9	1001	11	9
2	0010	2	2	10	1010	12	A
3	0011	3	3	11	1011	13	B
4	0100	4	4	12	1100	14	C
5	0101	5	5	13	1101	15	D
6	0110	6	6	14	1110	16	E
7	0111	7	7	15	1111	17	F

Decimal -> Binary (repeated division by 2)

```
2 | 25    -> remainder 1
2 | 12    -> remainder 0
2 |  6    -> remainder 0
2 |  3    -> remainder 1
2 |  1    -> remainder 1

                = 11001
```

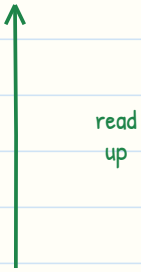


Fig 1: (25)10 = (11001)2 - read remainders bottom to top

- **Fraction:** multiply by 2 repeatedly, take the integer parts top to bottom. 0.625 -> 1.25(1), 0.5(0), 1.0(1) = **0.101**
- **Binary -> Decimal:** multiply each bit by its weight (powers of 2): 1101 = 8+4+0+1 = **13**

Octal & Hexadecimal shortcuts

1 0 1 1 1 0 1 0 1 1

Octal: 001|011|101|011

= 1 3 5 3 = (1353)8

Hex: 0010|1110|1011

= 2 E B = (2EB)16

Fig 2: group bits - 3 for octal, 4 for hex

Rule

Group from the RIGHT (the binary point for fractions: left of point groups from the point to the left, right side from the point to the right). Pad with 0s.

2. Fixed & Floating Point Representation

Fixed point: the binary point is at a fixed position (usually at the right end - integers, or after the sign bit - fractions). Simple, but small range.

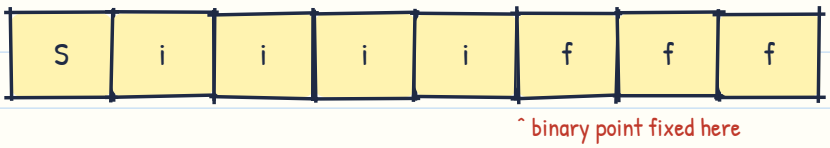


Fig 3: 8-bit fixed point (sign | 4 integer | 3 fraction)

Floating point: like scientific notation, number = $(-1)^S \times 1.M \times 2^{(E-bias)}$. Stores a wide range of values.

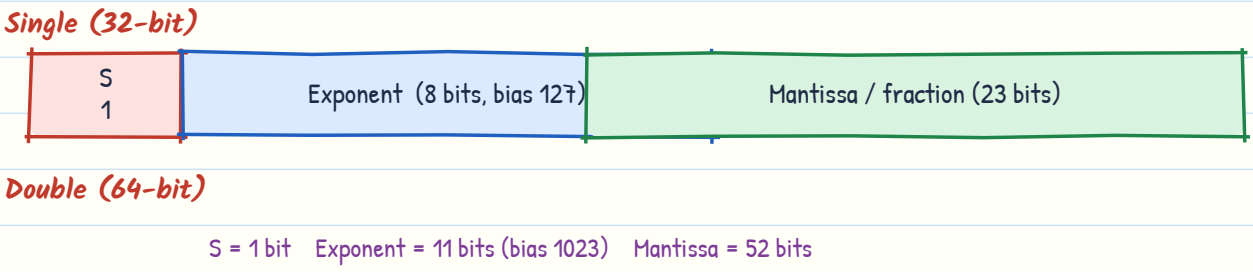


Fig 4: IEEE 754 formats

Example: store 12.5 in IEEE 754 single precision

Step	Work
1. Binary	12.5 = 1100.1
2. Normalize	1.1001 x 2^3 (move point 3 places)
3. Sign	positive -> S = 0
4. Exponent	3 + 127 = 130 = 10000010

5. Mantissa	1001 followed by 0s (hidden leading 1 dropped)
Result	0 10000010 10010000000000000000000 = 0x41480000

Remember

Normalized form always has a hidden leading 1, so only the digits AFTER the point (mantissa) are stored. Exponent is stored with a BIAS so negative exponents need no sign.

3. Signed Binary Numbers & Complements

To store negative numbers, the ***MSB is the sign bit*** (0 = positive, 1 = negative). Three methods:

Value	Sign-magnitude	1's complement	2's complement
+5	0101	0101	0101
-5	1101	1010	1011
+0	0000	0000	0000
-0	1000	1111	(none)
Range (4-bit)	-7 to +7	-7 to +7	-8 to +7

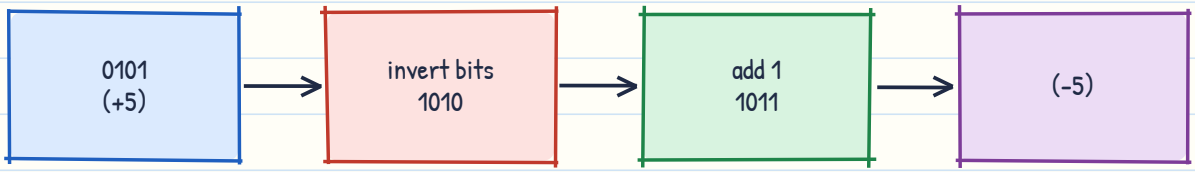


Fig 5: How to get 2's complement of 0101 (+5)

- ***1's complement*** = flip every bit. ***2's complement*** = 1's complement + 1.
- 2's complement is used in computers: only one zero and subtraction becomes addition.

Subtraction using 2's complement

```
  0111   (7)
+ 1011   (-5 in 2's)
-------
1 0010   -> discard carry

answer = 0010 = +2
```

Fig 6: 7 - 5 = 7 + (-5)

Carry vs overflow

If a carry goes out of the MSB it is DISCARDED. Overflow occurs when two numbers of the same sign give a result of the opposite sign.

r's and (r-1)'s complements

Base	(r-1)'s complement	r's complement
Binary (r=2)	1's complement	2's complement
Decimal (r=10)	9's complement	10's complement
Method	subtract each digit from r-1	(r-1)'s + 1

4. Binary Codes

Dec	BCD (8421)	Excess-3	Gray
0	0000	0011	0000
1	0001	0100	0001
2	0010	0101	0011
3	0011	0110	0010
4	0100	0111	0110
5	0101	1000	0111
6	0110	1001	0101
7	0111	1010	0100
8	1000	1011	1100
9	1001	1100	1101

- **BCD:** each decimal digit coded by 4 bits (1010–1111 unused). 25 = 0010 0101.
- **Excess-3:** BCD + 3; self-complementing code.
- **Gray code:** only ONE bit changes between consecutive values – used in encoders to avoid errors.
- **ASCII:** 7-bit code for characters (A = 65 = 1000001, a = 97, 0 = 48).

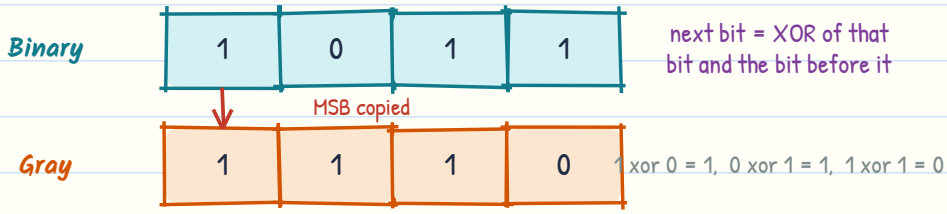


Fig 7: Binary -> Gray conversion (1011 -> 1110)

Error detection - parity bit

An extra bit is added so total 1s are **even (even parity)** or **odd**. Data 1011 has three 1s -> even-parity bit = **1** -> 11011. Detects any **single-bit** error.

5. Digital Logic Circuits - I

Basic logic functions & gates

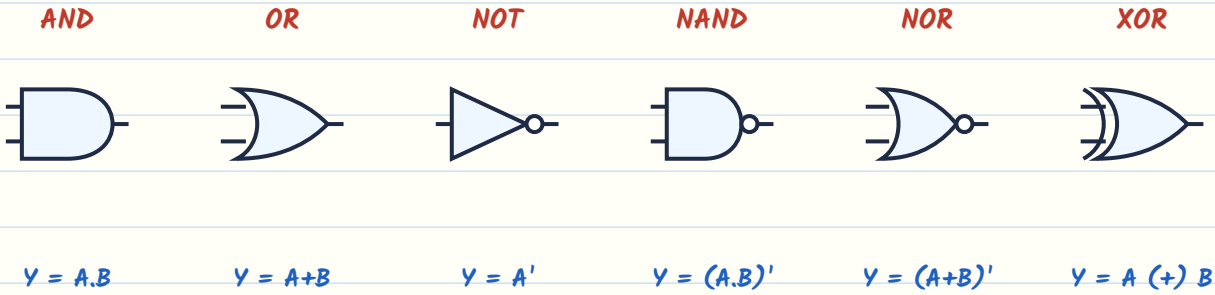


Fig 8: Logic gates and truth tables

A	B	AND	OR	NAND	NOR	XOR	XNOR	NOT A
0	0	0	0	1	1	0	1	1
0	1	0	1	1	0	1	0	1
1	0	0	1	1	0	1	0	0
1	1	1	1	0	0	0	1	0

- **AND:** output 1 only if ALL inputs are 1.
- **OR:** output 1 if ANY input is 1.
- **NOT:** inverts the input.
- **XOR:** 1 when inputs are DIFFERENT; **XNOR:** 1 when inputs are SAME.

Universal gates (NAND and NOR)

NAND and **NOR** are called ***universal gates*** because any logic function can be built using only one of them.

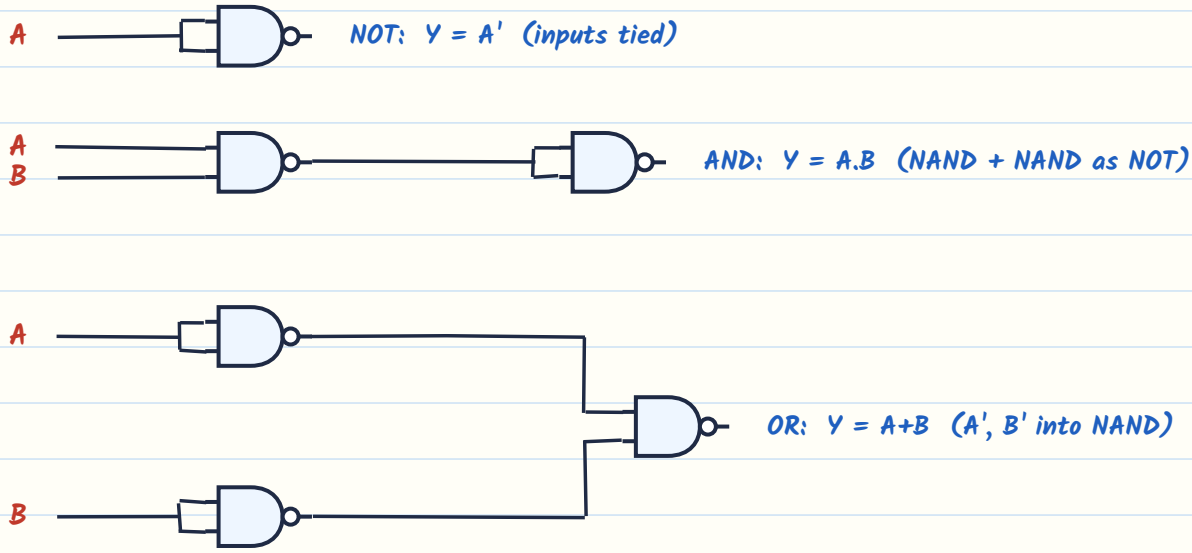


Fig 9: NOT, AND, OR built from NAND gates only

Required gate	Using NAND	Using NOR
NOT	NAND with inputs tied	NOR with inputs tied
AND	NAND -> NAND(as NOT)	invert both inputs, then NOR
OR	invert both inputs, then NAND	NOR -> NOR(as NOT)

Boolean algebra laws

Law	AND form	OR form
Identity	A.1 = A	A+0 = A
Null	A.0 = 0	A+1 = 1
Idempotent	A.A = A	A+A = A
Complement	$A.A' = 0$	$A+A' = 1$
Commutative	A.B = B.A	A+B = B+A
Distributive	A(B+C)=AB+AC	A+BC=(A+B)(A+C)
Absorption	A(A+B) = A	A+AB = A
De Morgan	$(AB)' = A'+B'$	$(A+B)' = A'B'$

Most asked!

De Morgan: "break the line, change the sign". NAND = OR of inverted inputs; NOR = AND of inverted inputs.

Minimization of logic expressions

Goal: fewer gates, cheaper and faster circuit. Forms: **SOP** (sum of products, minterms, e.g. AB + A'C) and **POS** (product of sums, maxterms).

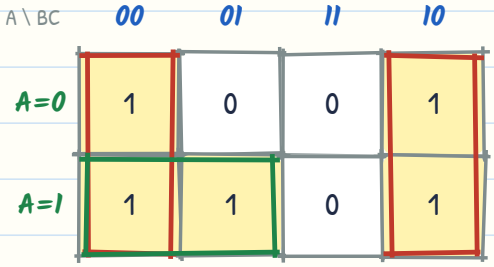
Algebraic simplification

```
Example:  F = AB + AB' + A'B
        = A(B + B') + A'B      [distributive]
        = A.1 + A'B            [complement]
        = A + A'B = A + B      [absorption variant]
```

Karnaugh map (K-map) simplification

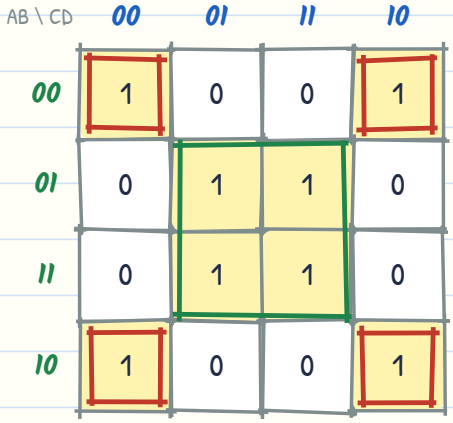
A K-map is a grid of minterms arranged in **Gray-code order** so that neighbouring cells differ by one variable. Group adjacent 1s in **powers of 2** (1,2,4,8,16).

- Groups must be rectangles (wrap-around allowed at edges); make groups as **big** as possible.
- Every 1 must be covered; overlap is allowed.
- A group eliminates the variables that change inside it.



quad (red) = C'
pair (green) = AB'

Fig 10: 3-variable K-map F = SUM m(0,2,4,5,6) -> F = C' + AB'



corners (red) = B'D'

centre (green) = BD

Fig 11: 4-variable K-map F = SUM m(0,2,5,7,8,10,13,15) -> F = B'D' + BD

Bonus tip

Don't-care conditions (X) may be used as 1 or 0 - whichever gives bigger groups.

6. Combinational Circuits

Output depends ***only on present inputs*** (no memory). Examples: adders, decoders, multiplexers, encoders.

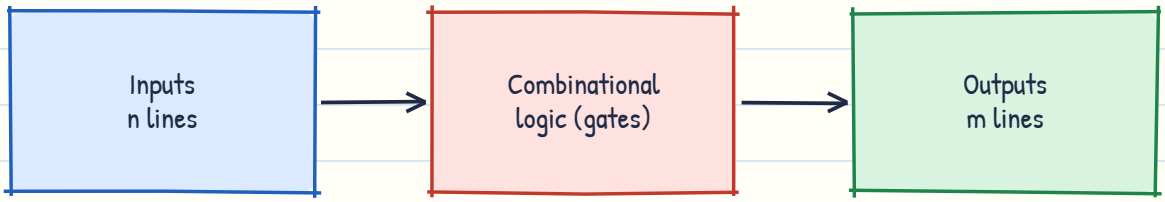


Fig 12: Combinational circuit block

Half adder & full adder

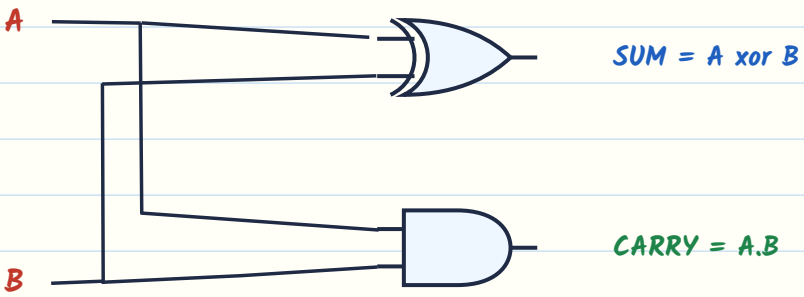


Fig 13: Half adder (adds two bits)

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

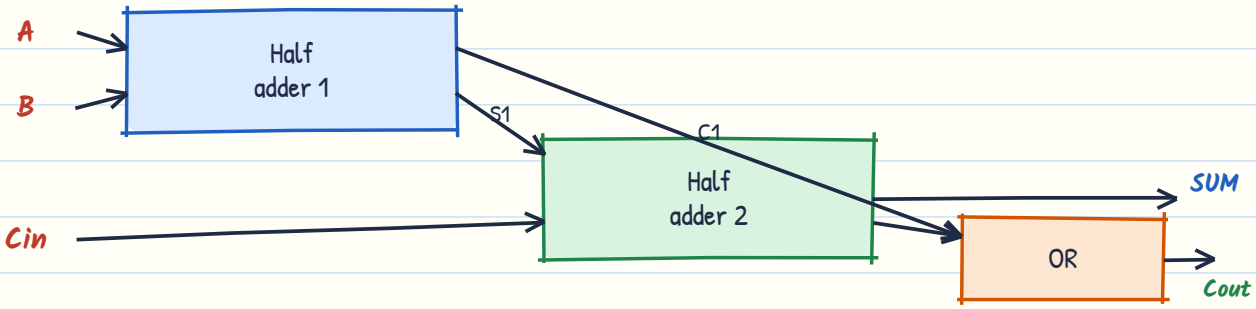


Fig 14: Full adder = 2 half adders + OR gate

- Sum = A (+) B (+) Cin Cout = AB + Cin(A (+) B)

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Decoder

A *decoder* converts an n-bit code into one of 2^n output lines (only one output active). Use: address decoding, memory chip select.

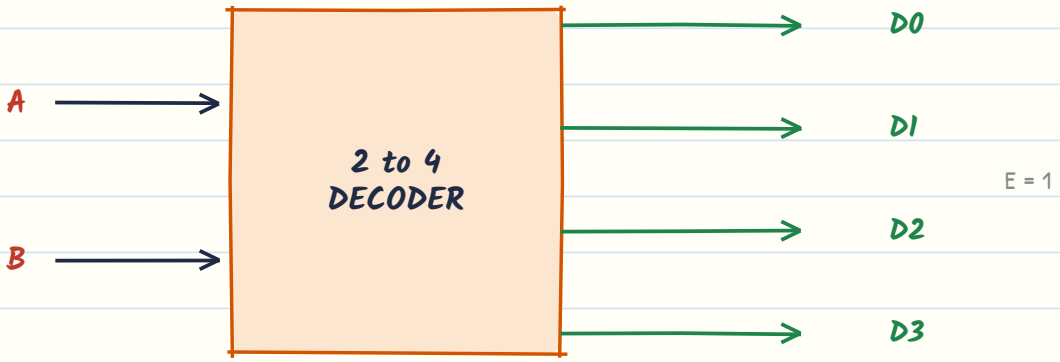


Fig 15: 2-to-4 decoder

A	B	D0	D1	D2	D3
0	0	1	0	0	0
0	1	0	1	0	0

1	0	0	0	1	0
1	1	0	0	0	1

Each output is a ***minterm***: D0 = A'B', D1 = A'B, D2 = AB', D3 = AB. A ***3-to-8 decoder*** has 3 inputs and 8 outputs; two 3-to-8 decoders can build a 4-to-16 decoder (using the enable pin).

Multiplexer (MUX) - data selector

A ***multiplexer*** selects ***one of many inputs*** and sends it to a single output using select lines. 2^n inputs need n select lines.

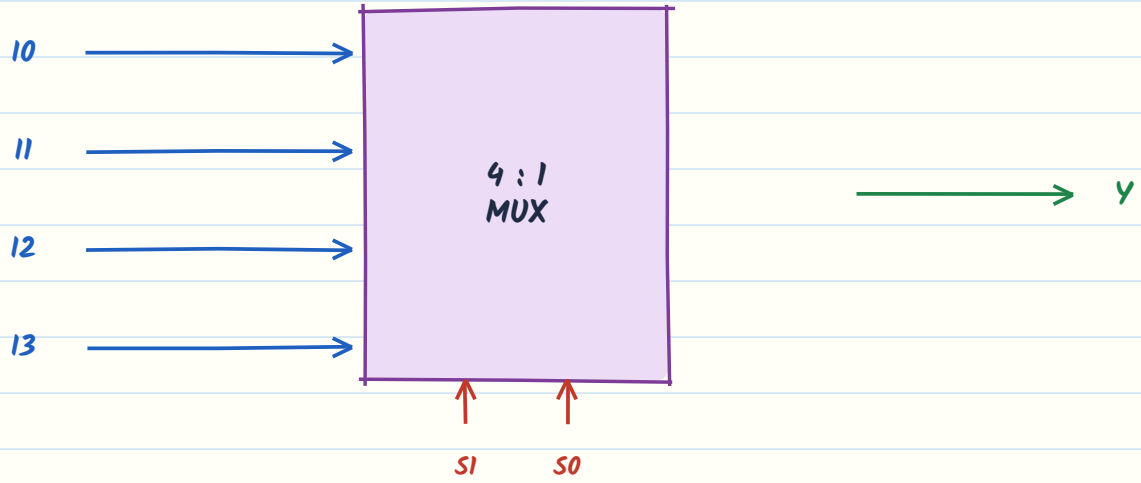


Fig 16: 4-to-1 multiplexer

S1	S0	Y
0	0	I0
0	1	I1
1	0	I2
1	1	I3

- ***Y = S1'S0' I0 + S1'S0 I1 + S1S0' I2 + S1S0 I3***
- Larger MUX from smaller: two 4:1 + one 2:1 = 8:1.
- A MUX can implement any Boolean function (use inputs as 0/1 constants).

Quick Revision & Exam Questions

Topic	Memory line
Conversions	Dec->Bin: divide by 2; Bin->Oct/Hex: group 3/4 bits
IEEE 754	1 | 8 | 23 bits, bias 127, hidden 1
2's complement	invert + 1; range -8 to +7 (4-bit)
Gray code	one bit changes at a time

Universal gates	NAND, NOR
De Morgan	(AB)'=A'+B'; (A+B)'=A'B'
Decoder / MUX	n -> 2^n lines / 2^n inputs -> 1 output

Likely exam questions

- Convert numbers between binary, octal, decimal and hexadecimal.
- Explain fixed and floating point representation (IEEE 754).
- Explain signed number representations; subtract using 2's complement.
- Explain binary codes: BCD, Excess-3, Gray, ASCII.
- Why are NAND and NOR universal? Realize AND, OR, NOT using NAND.
- Simplify a 3/4-variable function with K-map.
- Design a full adder; explain decoder and multiplexer.